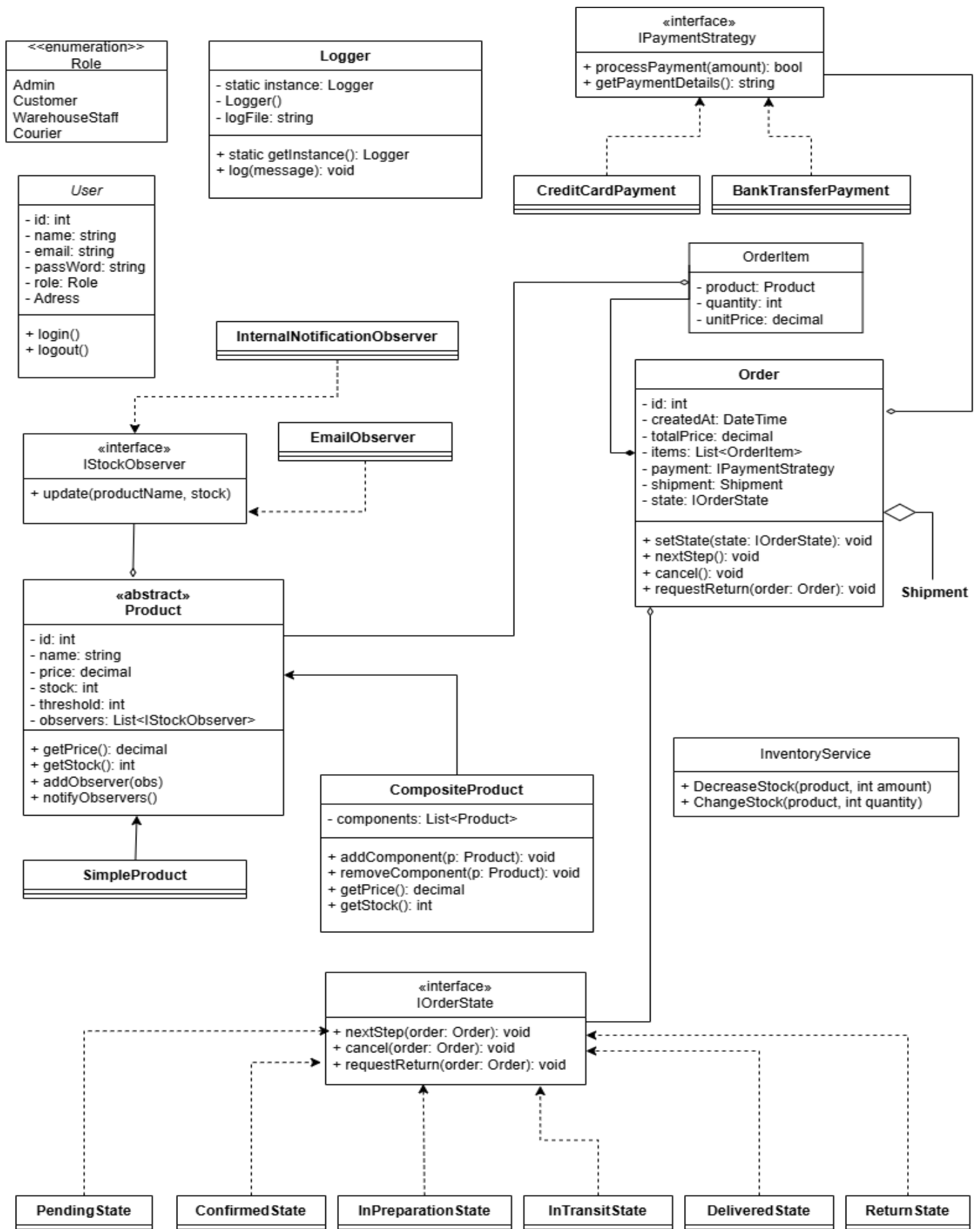
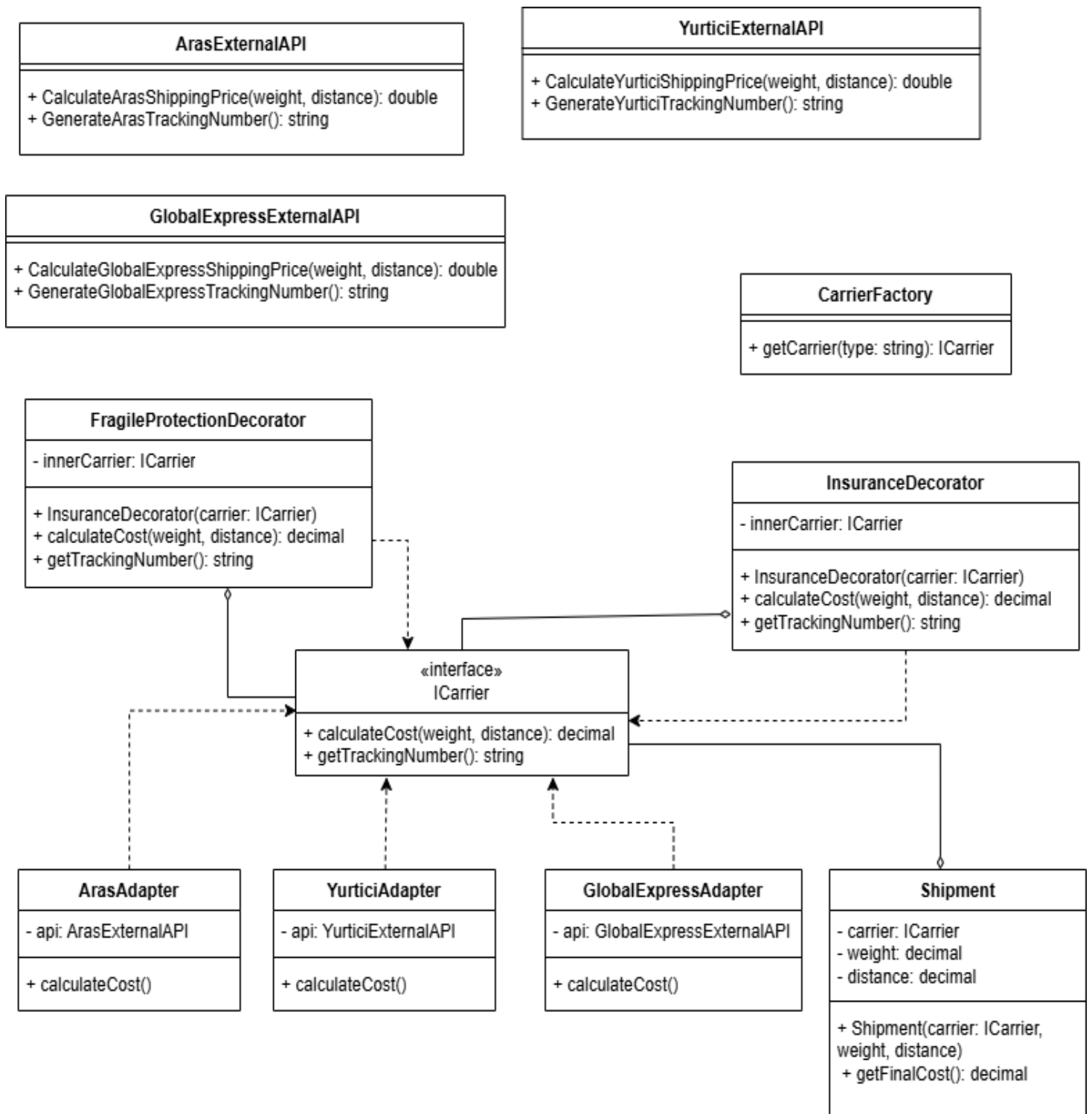


Adı Soyadı: LUCAS ISAAC CASSOMA KATALAHALI – 1A Grubu





Sınıf Diyagramlarının Analizi: Tedarik ve Lojistik Sistemi

1. Temel Mimari ve Tasarım Desenleri

Sistem, ölçeklenebilirlik ve bakım kolaylığı sağlamak için modüler bir mimari üzerine kurulmuştur ve çeşitli **Design Pattern**'ler kullanılmıştır:

- **Composite Pattern (Ürünler):** Soyut Product sınıfı, SimpleProduct (Basit Ürün) ve CompositeProduct (Birleşik Ürün) için temel görevi görür. Bu yapı sayesinde sistem, tek ürünleri ve "Pack" (ürün kombinasyonları) yapılarını aynı şekilde yönetebilir. CompositeProduct, fiyatını ve stok miktarını içindeki bileşenlere göre dinamik olarak hesaplar.
- **State Pattern (Sipariş Akışı):** Order sınıfı davranışlarını IOrderState arayüzüne devreder. Somut durum sınıfları (Pending, InTransit, Delivered gibi) geçişleri ve her aşamada yapılabilecek işlemleri yönetir.
- **Strategy Pattern (Ödemeler):** Ödeme yöntemleri, IPaymentStrategy arayüzü sayesinde Order sınıfından ayrılmıştır. Böylece yeni ödeme yöntemleri (Kredi Kartı, Banka Havalesi gibi) mevcut sipariş mantığını değiştirmeden eklenebilir.
- **Observer Pattern (Envanter):** IStockObserver arayüzü, bağımsız bir bildirim sistemi sağlar. Stok seviyeleri değiştiğinde EmailObserver ve InternalNotificationObserver otomatik olarak bilgilendirilir ve gerekli uyarıları gönderir.

2. Lojistik ve Harici Sistem Entegrasyonu

İkinci diyagram, gönderim modülünün genişletilebilir yapısına odaklanmaktadır:

- **Adapter Pattern (Kargo Firmaları):** Farklı lojistik API'leriyle (Aras, Yurtiçi, GlobalExpress) entegrasyon sağlamak için **Adapter Pattern** kullanılmıştır. Böylece farklı API çağrıları standart bir ICarrier arayüzü altında birleştirilmiş ve sistem dış servislerdeki değişikliklerden korunmuştur.
- **Decorator Pattern (Gönderim Özellikleri):** FragileProtection (Kırılabilir Koruması) ve Insurance (Sigorta) gibi ek gönderim hizmetleri, ICarrier için dekoratör olarak uygulanmıştır. Bu yapı sayesinde sisteme dinamik olarak ek maliyetler ve ek davranışlar eklenebilir, temel kargo sınıfları değiştirilmez.
- **Factory Pattern:** CarrierFactory sınıfı nesne oluşturma mantığını kapsüller ve kullanıcı seçimine veya iş kurallarına göre uygun kargo firmasını oluşturur.

3. Temel İlişkiler ve Altyapı

- **Singleton Pattern:** Logger sınıfı özel bir constructor ve statik getInstance() metodu kullanır. Böylece sistem logları için tek bir global erişim noktası sağlanır ve dosya erişim çakışmaları önlenir.
- **Veri Kalıcılığı ve Roller (Roles):** User varlığı, **Role-Based Access Control (RBAC)** yapısını yönetmek için Role enum'unu kullanır. Böylece stok yenileme veya sipariş

gönderme gibi hassas işlemler sadece yetkili personel (WarehouseStaff, Courier) tarafından yapılabilir.